

UNIVERSIDAD AUTONOMA DE AGUASCALIENTES

**INGENIERIA EN COMPUTACION
INTELIGENTE**

**ADMINISTRACION DE SOFTWARE Y
PROYECTOS**



PLANIFICACION SPRINT 1

Arrioja Pizaña Cesar Emmanuel

Cruz Mora Ángel

Semestre VIII

Interfaz Móvil y Vista de Ruta para el Repartidor

Este sprint se enfocó de manera integral en el brazo operativo en campo: los conductores o repartidores (*Drivers*). Con el objetivo de eliminar el uso de papel en los camiones de distribución, el desarrollo se centró en una interfaz optimizada para dispositivos móviles ("Mobile-First"). A nivel de código, se diseñó la lógica para capturar las órdenes que ya han sido procesadas por la oficina y presentarlas al operador de forma limpia y directa. El avance se tradujo en una vista funcional donde el conductor puede revisar su itinerario del día y los detalles específicos de los activos que debe trasladar, eliminando las pérdidas o daños asociados a los formatos físicos.

Durante esta primera iteración, se construyeron los cimientos de software y seguridad del sistema, avanzando prioritariamente en el flujo de verificación y la conexión asíncrona del operario. Utilizando una arquitectura moderna basada en *React* ("*use client*"), se integró el proveedor de identidad **Clerk** para mitigar riesgos de accesos no autorizados mediante tokens de sesión dinámicos. El desarrollo incluyó la creación del panel del conductor (DriverDashboard), el cual ejecuta una doble validación: primero autentica la identidad del usuario y posteriormente consulta de forma segura el rol en la base de datos PostgreSQL mediante el *gateway* `/api/users`. Si el usuario no posee el rol explícito de DRIVER, el sistema deniega el acceso rediriéndolo de inmediato a la raíz.

Una vez superado el bloqueo de seguridad, el código inicializa de manera automática un flujo de datos bidireccional continuo mediante **Server-Sent Events (SSE)** conectado al broker `/api/events`. Esta implementación remueve por completo la necesidad de consultas manuales repetitivas, permitiendo que el dispositivo del conductor permanezca a la escucha de alertas críticas de despacho (como la asignación de nuevas órdenes por parte de la oficina) con un consumo mínimo de red en campo.

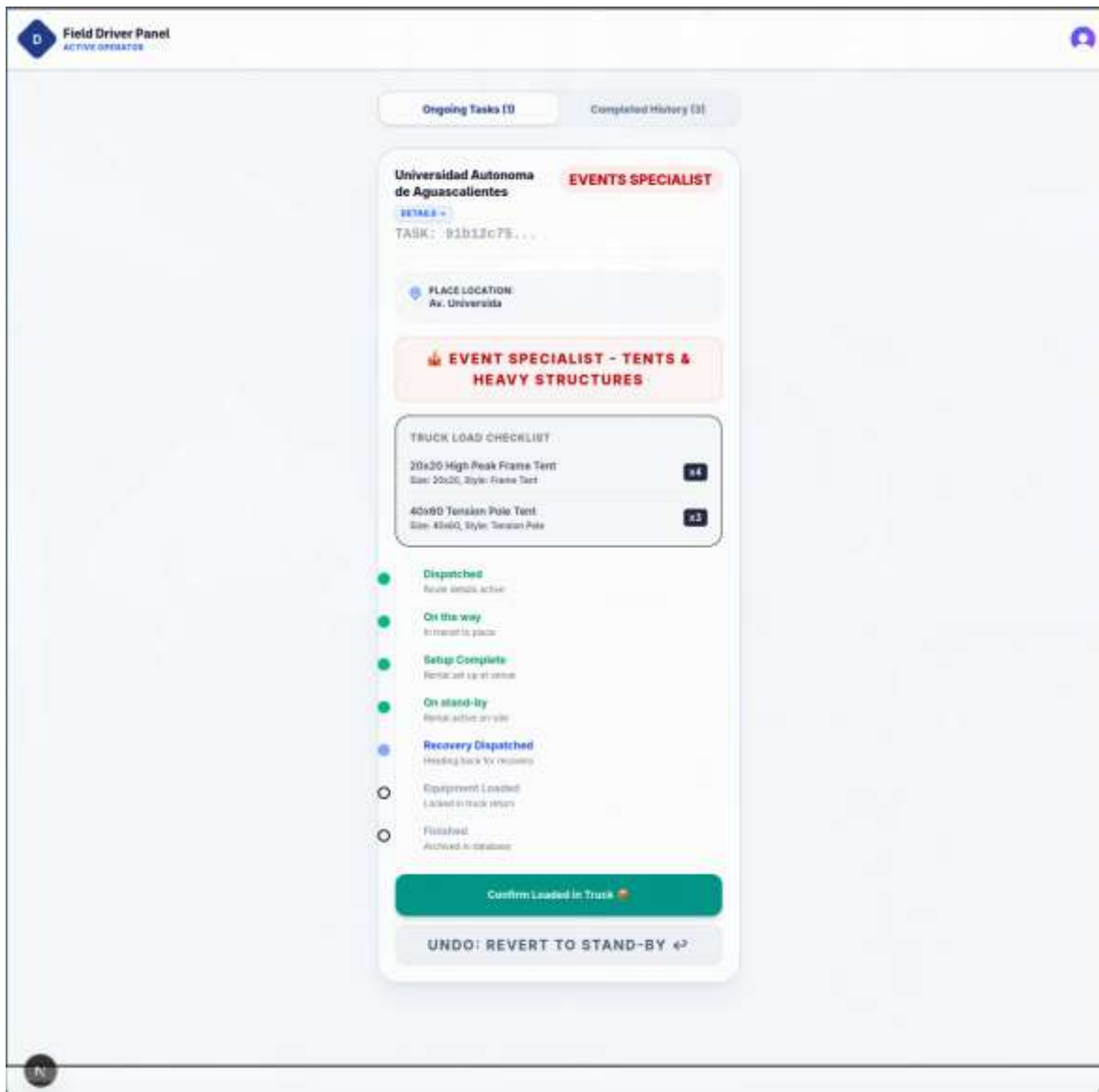
LOGROS ALCANZADOS (A GRANDES RASGOS): Se desarrolló la *Vista de Ruta para Conductores*, optimizando el renderizado para pantallas de teléfonos inteligentes.

- Se codificó el **Módulo de Login e Identificación** aprovechando el hook `useUser` del SDK oficial de Clerk para validar credenciales activas de operador.
- Se implementaron los controladores de estado reactivos (`authorized`, `loading`) para manejar el ciclo de vida del aprovisionamiento y bloquear el renderizado de datos sensibles durante la verificación.
- Se programó la función asíncrona `fetchAssignments` para recuperar el listado inicial de contratos digitales desde la API `/api/orders`, ordenando las tareas activas sobre el histórico de completadas.

- Se estableció el canal de streaming de eventos continuo mediante la API nativa de navegadores EventSource, enlazando el tópico tasks.lifecycle.assigned para inyectar notificaciones visuales e instantáneas de la oficina al repartidor.
- Se definió la máquina de estados e interfaces TypeScript (InventoryItem, Order) que gobernarán la consistencia del flujo de activos y la segmentación funcional (*Events* vs *Party*) en las siguientes fases de desarrollo.

ENTREGABLES

Evidencia:



Código realizado:

```
import { useEffect, useState } from "react";
import { UserButton, useUser } from "@clerk/nextjs";
import Link from "next/link";

interface InventoryItem {
  id: number;
  quantity: number;
  sku: string;
  name: string;
  category: string;
  detail?: string;
}

interface Order {
  id: string;
  division: "EVENTS" | "PARTY";
  status: string;
  pickupDate: string | null;
  createdAt: string;
  updatedAt: string;
  clientName: string | null;
  clientPhone: string | null;
  clientEmail: string | null;
  clientAddress: string | null; // place
  items: InventoryItem[];
}

export default function DriverDashboard() {
  const { user: clerkUser } = useUser();
  const [authorized, setAuthorized] = useState<boolean | null>(null);

  const [activeOrders, setActiveOrders] = useState<Order[]>([]);
  const [loading, setLoading] = useState(true);
  const [updatingId, setUpdatingId] = useState<string | null>(null);
  const [notification, setNotification] = useState<string | null>(null);
  const [itemChecks, setItemChecks] = useState<{ [orderId: string]: { [itemId: number]: boolean } }>({});
  const [driverTab, setDriverTab] = useState<"active" | "completed">("active");
  const [modal, setModal] = useState<{
    isOpen: boolean;
    title: string;
    message: string;
    type: "alert" | "confirm";
    onConfirm?: () => void;
  }>({
    isOpen: false,
    title: "",
    message: "",
    type: "alert"
  });

  const triggerAlert = (title: string, message: string) => {
    setModal({
      isOpen: true,
      title,
      message,
      type: "alert",
      onConfirm: () => {
        // handle confirm logic
      }
    });
  };
}
```

```

    message,
    type: "alert"
  });
};

const triggerConfirm = (title: string, message: string, onConfirm: () => void) => {
  setModal({
    isOpen: true,
    title,
    message,
    type: "confirm",
    onConfirm
  });
};

```

```

// Fetch driver assignments from Postgres
const fetchAssignments = async (driverClerkId: string) => {
  try {
    const res = await fetch(`/api/orders?userId=${driverClerkId}&role=DRIVER`);
    const data = await res.json();
    if (data.success) {
      // Render incomplete/active tasks first, then completed ones
      const list = data.activeOrders || [];
      const sortedList = [
        ...list.filter((o: Order) => o.status !== "COMPLETED"),
        ...list.filter((o: Order) => o.status === "COMPLETED")
      ];
      setActiveOrders(sortedList);
    }
  } catch (err) {
    console.error("Driver fetch assignments failed:", err);
  } finally {
    setLoading(false);
  }
};

```

```

// Perform dynamic authorization checks
useEffect(() => {
  if (!clerkUser) return;
  const userId = clerkUser.id;

  async function checkAuthorization() {
    try {
      const res = await fetch(`/api/users?userId=${userId}`);
      const data = await res.json();
      if (data.success && data.user && data.user.role === "DRIVER") {
        setAuthorized(true);
      } else {
        setAuthorized(false);
        window.location.href = "/";
      }
    } catch (err) {
      console.error("Authorization check failed:", err);
      setAuthorized(false);
      window.location.href = "/";
    }
  }
}
);

```

```

    }
    checkAuthorization();
  }, [clerkUser]);

// Set up SSE stream once authorized and clerkUser loaded
useEffect(() => {
  if (authorized !== true || !clerkUser) return;

  const driverId = clerkUser.id;
  setLoading(true);
  fetchAssignments(driverId);

  // Register SSE Stream connection for live notifications (e.g. when office assigns new tasks!)
  console.log(`[Driver SSE] Connecting for driver userId=${driverId}...`);
  const eventSource = new EventSource(`/api/events?userId=${driverId}&role=DRIVER`);

  eventSource.onmessage = (event) => {
    try {
      const data = JSON.parse(event.data);
      if (data.type === "connection") return;

      console.log("[Driver SSE] Stream update received:", data);

      if (data.topic === "tasks.lifecycle.assigned") {
        setNotification(`Dispatch Alert: Office assigned you order for`
          `${data.event.payload.clientName}!`);
        setTimeout(() => setNotification(null), 6000);
      }

      // Refresh state
      fetchAssignments(driverId);

    } catch (err) {
      console.error("[Driver SSE] Parse error:", err);
    }
  };

  eventSource.onerror = () => {
    console.warn("[Driver SSE] Connection interrupted. Stream retrying...");
  };

  return () => {
    console.log("[Driver SSE] Closing stream...");
    eventSource.close();
  };
}, [authorized, clerkUser]);

const handleStatusChange = async (orderId: string, newStatus: string) => {
  if (!clerkUser) return;
  const driverId = clerkUser.id;

  try {
    setUpdatingId(orderId);

    let url = "/api/orders/progress";
    let body: any = { orderId, driverId, newStatus };

```

```

// If finalizing the task, route to complete API
if (newStatus === "COMPLETED") {
  url = "/api/orders/complete";
  body = { orderId, driverId };
}

const res = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(body)
});
const data = await res.json();

if (data.success) {
  fetchAssignments(driverId);
} else {
  triggerAlert("Operations Gateway Warning", data.error || "Failed to update task status.");
}
} catch (err) {
  console.error("Status transition error:", err);
  triggerAlert("Connection Gateway Failure", "Error communicating with dispatch operations broker.");
} finally {
  setUpdatingId(null);
}
};

// Helper to map double phase states to actions
const getNextAction = (status: string) => {
  switch (status) {
    case "ASSIGNED":
      return { label: "Confirm Load & Scan Truck ", nextStatus: "OUT_FOR_DELIVERY", style: "bg-amber-600 hover:bg-amber-500 shadow-amber-950/20" };
    case "OUT_FOR_DELIVERY":
      return { label: "Deliver & Complete Setup ", nextStatus: "DELIVERED", style: "bg-cyan-600 hover:bg-cyan-500 shadow-cyan-950/20" };
    case "DELIVERED":
      return { label: "Complete Setup & Enter Standby 🕒 ", nextStatus: "STANDBY", style: "bg-indigo-600 hover:bg-indigo-500 shadow-indigo-950/20" };
    case "STANDBY":
      return { label: "Initiate Return Recovery ", nextStatus: "OUT_FOR_PICKUP", style: "bg-orange-600 hover:bg-orange-500 shadow-orange-950/20" };
    case "OUT_FOR_PICKUP":
      return { label: "Confirm Loaded in Truck ", nextStatus: "PICKED_UP", style: "bg-teal-600 hover:bg-teal-500 shadow-teal-950/20" };
    case "PICKED_UP":
      return { label: "Finalize Delivery & Archive ", nextStatus: "COMPLETED", style: "bg-emerald-600 hover:bg-emerald-500 shadow-emerald-950/20" };
    default:
      return null;
  }
};

// Helper to map double phase states to undo options
const getPreviousStatus = (status: string) => {
  switch (status) {

```

```

case "OUT_FOR_DELIVERY":
  return { label: "Revert to Dispatched ⇐", prevStatus: "ASSIGNED" };
case "DELIVERED":
  return { label: "Revert to On the Way ⇐", prevStatus: "OUT_FOR_DELIVERY" };
case "STANDBY":
  return { label: "Revert to Setup/Delivering ⇐", prevStatus: "DELIVERED" };
case "OUT_FOR_PICKUP":
  return { label: "Revert to Stand-by ⇐", prevStatus: "STANDBY" };
case "PICKED_UP":
  return { label: "Revert to Recovery Dispatched ⇐", prevStatus: "OUT_FOR_PICKUP" };
default:
  return null;
}
};

// Render vertical timeline steps
const renderTimeline = (currentStatus: string) => {
  const steps = [
    { key: "ASSIGNED", label: "Dispatched", desc: "Route details active" },
    { key: "OUT_FOR_DELIVERY", label: "On the way", desc: "In transit to place" },
    { key: "DELIVERED", label: "Setup Complete", desc: "Rental set up at venue" },
    { key: "STANDBY", label: "On stand-by", desc: "Rental active on-site" },
    { key: "OUT_FOR_PICKUP", label: "Recovery Dispatched", desc: "Heading back for recovery" },
    { key: "PICKED_UP", label: "Equipment Loaded", desc: "Locked in truck return" },
    { key: "COMPLETED", label: "Finished", desc: "Archived in database" }
  ];

  const getStatusIndex = (status: string) => {
    const idx = steps.findIndex(s => s.key === status);
    return idx === -1 ? 0 : idx;
  };

  const activeIndex = getStatusIndex(currentStatus);

  return (
    <div className="space-y-4 my-6 pl-2 relative border-l border-[#E2E8F0]">
      {steps.map((step, idx) => {
        const isDone = idx < activeIndex || currentStatus === "COMPLETED";
        const isCurrent = idx === activeIndex && currentStatus !== "COMPLETED";

        return (
          <div key={step.key} className="relative pl-6">
            {/* Timeline dot */}
            <span className={`absolute left-[-29px] top-1.5 h-3.5 w-3.5 rounded-full border-2 ${
              isDone
                ? "bg-emerald-500 border-emerald-500 shadow-lg shadow-emerald-500/20"
                : isCurrent
                ? "bg-blue-600 border-blue-600 animate-pulse shadow-lg shadow-blue-500/20"
                : "bg-white border-slate-350"
            }` />
          </div>
          <div>
            <h5 className={`text-xs font-bold ${
              isDone
                ? "text-emerald-600"
                : isCurrent
            }` />
          </div>
        );
      })}
    </div>
  );

```



```

        ? "text-blue-700"
        : "text-slate-400"
    }`>
    {step.label}
</h5>
    <p className="text-[10px] text-gray-500 mt-0.5">{step.desc}</p>
</div>
</div>
);
}}
</div>
);
};

if (authorized === null) {
    return (
        <div className="min-h-screen bg-[#0B2545] text-white flex flex-col items-center justify-center p-6 font-sans">
            <div className="flex flex-col items-center gap-4">
                <div className="h-10 w-10 border-4 border-blue-500 border-t-transparent rounded-full animate-spin" />
                <p className="text-xs text-slate-350 tracking-wider uppercase font-bold">Verifying System Access Credentials...</p>
            </div>
        </div>
    );
}

if (authorized !== true) {
    return null;
}

return (
    <div className="min-h-screen bg-[#F8FAFC] text-[#0F172A] pb-12 font-sans relative flex flex-col">

        {/* Live notification stream alert banner */}
        {notification && (
            <div className="fixed top-4 left-4 right-4 z-50 bg-blue-50 border-2 border-blue-600/40 p-4 rounded-xl shadow-xl backdrop-blur-md animate-bounce flex items-center justify-between text-xs font-bold text-blue-900">
                <span>{notification}</span>
                <button onClick={() => setNotification(null)} className="text-blue-900 font-extrabold ml-2 text-sm">x</button>
            </div>
        )}

        {/* Header Bar */}
        <header className="bg-white border-b border-[#E2E8F0] shadow-sm p-4 sticky top-0 z-45 backdrop-blur-md flex items-center justify-between">
            <div className="flex items-center gap-3">
                <Link href="/" className="h-9 w-9 bg-[#1E3A8A] rounded-lg flex items-center justify-center font-bold text-white shadow active:scale-95 transition-all text-sm transform rotate-45">
                    <span className="transform -rotate-45">D</span>
                </Link>
            </div>
            <h1 className="text-sm font-extrabold text-slate-900">Field Driver Panel</h1>

```

```

    <p className="text-[9px] text-[#2563EB] tracking-wider font-extrabold uppercase mt-[-1px]">
      {clerkUser?.fullName || "Active Operator"}
    </p>
  </div>
</div>

<div className="flex items-center gap-3">
  <UserButton />
</div>
</header>

{/* Main Body */}
<main className="max-w-md w-full mx-auto p-4 flex-1 flex flex-col gap-6 mt-2">
  {/* Sections Selector Tabs: Ongoing vs Completed */}
  <div className="flex bg-[#F1F5F9] border border-[#E2E8F0] p-0.5 rounded-2xl shrink-0">
    <button
      onClick={() => setDriverTab("active")}
      className={`w-1/2 py-2.5 rounded-xl text-xs font-extrabold transition-all flex items-center justify-
center gap-1.5 ${
        driverTab === "active"
          ? "bg-white text-blue-900 shadow-sm"
          : "text-slate-500 hover:text-slate-900"
        }`}>
      >
      Ongoing Tasks ({activeOrders.filter(o => o.status !== "COMPLETED").length})
    </button>
    <button
      onClick={() => setDriverTab("completed")}
      className={`w-1/2 py-2.5 rounded-xl text-xs font-extrabold transition-all flex items-center justify-
center gap-1.5 ${
        driverTab === "completed"
          ? "bg-white text-blue-900 shadow-sm"
          : "text-slate-500 hover:text-slate-900"
        }`}>
      >
      Completed History ({activeOrders.filter(o => o.status === "COMPLETED").length})
    </button>
  </div>

  {loading ? (
    <div className="text-center py-20 text-xs text-gray-500 font-medium">Reading
assignments...</div>
  ) : activeOrders.filter(o => driverTab === "active" ? o.status !== "COMPLETED" : o.status ===
"COMPLETED").length === 0 ? (
    <div className="text-center py-24 border-2 border-dashed border-[#E2E8F0] rounded-3xl bg-white
p-6 shadow-sm flex flex-col items-center justify-center">
      <svg className="h-10 w-10 text-slate-400 mb-3" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={1.5} d="M9 5H7a2 2 0 0 2
2v12a2 2 0 0 2 2h10a2 2 0 0 2 2V7a2 2 0 0 2 2h-2M9 5a2 2 0 0 2 2h2a2 2 0 0 2 2M9 5a2 2 0 0 2
2h2a2 2 0 0 2 2" />
      </svg>
      <h4 className="text-sm font-bold text-slate-800">
        {driverTab === "active" ? "No ongoing dispatches!" : "Archive empty!"}
      </h4>
    <p className="text-xs text-gray-500 mt-1 max-w-[220px] text-center">

```

```

        {driverTab === "active"
          ? "You have no active route tasks assigned. New tasks will pop here instantly."
          : "No completed assignments found in your historical archive."}
      </p>
    </div>
  ) : (
    <div className="space-y-6">
      {activeOrders
        .filter(o => driverTab === "active" ? o.status !== "COMPLETED" : o.status ===
"COMPLETED")
        .map((order) => {
          const action = getNextAction(order.status);
          const isFinished = order.status === "COMPLETED";
          return (
            <div
              key={order.id}
              className={`bg-white border rounded-3xl p-5 transition-all ${
                isFinished
                  ? "border-slate-200/60 opacity-60 shadow-sm bg-slate-50/20"
                  : "border-blue-200 shadow-xl shadow-blue-900/5 bg-white relative overflow-hidden"
              }`}
            >

              {/* Top Header Section (Clickable to view detailed map & phase details) */}
              <Link
                href={`/${driver}/order/${order.id}`}
                className="flex items-start justify-between gap-3 border-b border-[#F1F5F9] pb-3.5 mb-4
hover:opacity-85 transition-opacity block group cursor-pointer"
              >
                <div>
                  <h4 className="font-extrabold text-slate-900 text-sm leading-snug group-hover:text-blue-
700 transition-colors flex items-center gap-1.5 flex-wrap">
                    {order.clientName}
                    <span className="text-[9px] font-extrabold text-blue-600 bg-blue-50 border border-blue-
100 px-1.5 py-0.5 rounded uppercase tracking-wider group-hover:bg-blue-600 group-hover:text-white
transition-all shadow-sm">
                      Details →
                    </span>
                  </h4>
                  <p className="text-3xs text-gray-400 font-mono font-bold mt-0.5">TASK:
{order.id.slice(0, 8)} ...</p>
                </div>

                <div className="flex flex-col items-end gap-1 shrink-0">
                  <span className={`px-2.5 py-0.5 rounded-full text-3xs font-extrabold uppercase ${
                    order.division === "EVENTS"
                      ? "bg-red-50 text-red-700 border border-red-100"
                      : "bg-blue-50 text-blue-700 border border-blue-100"
                  }`} >
                    {order.division} Specialist
                  </span>
                </div>
              </Link>

              {/* Active Order Focus Details: Client phone and Place address */}
              {!isFinished && (

```

```

<div className="mb-4 space-y-2.5 bg-slate-50 border border-slate-100 p-3.5 rounded-2xl
text-xs text-slate-700">
  {/ * Address Location place */}
  <div className="flex items-start gap-2">
    <svg className="h-4 w-4 text-blue-600 shrink-0 mt-0.5" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M17.657
16.657L13.414 20.9a1.998 1.998 0 01-2.827 0l-4.244-4.243a8 8 0 1111.314 0z" />
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M15 11a3 3 0
11-6 0 3 3 0 016 0z" />
    </svg>
    <div>
      <span className="font-bold text-slate-800 text-[10px] uppercase tracking-wider
block">Place Location:</span>
      <span className="font-semibold text-slate-800 mt-0.5">{order.clientAddress}</span>
    </div>
  </div>

  {/ * Phone contact */}
  {order.clientPhone && (
    <div className="flex items-center gap-2 border-t border-slate-200/50 pt-2">
      <svg className="h-4 w-4 text-blue-600 shrink-0" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M3 5a2 2 0
012-2h3.28a1 1 0 01.94.725l.548 2.2a1 1 0 01-.321.988l-1.305.98a10.582 10.582 0 004.872 4.872l.98-
1.305a1 1 0 01.988-.321l2.2.548a1 1 0 01.725.94V19a2 2 0 01-2 2h-1C9.716 21 3 14.284 3 6V5z" />
      </svg>
      <div>
        <span className="font-bold text-slate-800 text-[10px] uppercase tracking-wider
block">Contact Phone:</span>
        <a
          href={`tel:${order.clientPhone}`}
          className="font-bold text-blue-700 hover:text-blue-800 underline block mt-0.5
transition-colors"
          >
          {order.clientPhone} (Tap to Call)
        </a>
      </div>
    </div>
  )}
</div>
)}

{/ * Standby phase indicator */}
{!isFinished && order.status === "STANDBY" && (
  <div className="mb-4 bg-indigo-50/70 border border-indigo-100 p-3.5 rounded-2xl text-xs
flex flex-col gap-1.5 shadow-sm text-slate-700 animate-in fade-in slide-in-from-top-2 duration-200">
    <span className="text-[10px] font-extrabold uppercase tracking-widest text-indigo-600
block font-sans">
      On Stand-by Telemetry
    </span>
    <div className="flex items-center gap-2 font-extrabold text-slate-800 text-sm">
      <span> Pick-up Date:</span>
      <span className="bg-indigo-600 text-white px-2.5 py-0.5 rounded-full text-xs font-
mono">

```

```

        {order.pickupDate ? new Date(order.pickupDate + "T00:00:00").toLocaleDateString("en-US", { dateStyle: "long" }) : "Not Specified"}
      </span>
    </div>
    <p className="text-[10px] text-indigo-600/80 leading-relaxed font-bold mt-1">
      Equipment is currently operating on-site at the venue. Recovery will be re-dispatched on the
      scheduled pick-up date.
    </p>
  </div>
)}

{/* Specialist Banner so client identifies role */}
{!isFinished && (
  <div className={`p-3 rounded-xl border text-center text-3xs font-extrabold tracking-widest
    uppercase mb-4 ${
      order.division === "EVENTS"
        ? "bg-red-50/50 border-red-200 text-red-700"
        : "bg-blue-50/50 border-blue-200 text-blue-700"
    }`} >
    {order.division === "EVENTS"
      ? " EVENT SPECIALIST - TENTS & HEAVY STRUCTURES"
      : " PARTY SPECIALIST - FINE FURNITURE & BANQUET"
    }
  </div>
)}

{/* Load Manifest Load items */}
<div className="bg-slate-50 border border-slate-150 rounded-2xl p-4 text-xs">
  <span className="text-gray-500 font-bold uppercase tracking-wider text-3xs block mb-2.5">
    {order.status === "DELIVERED" ? "Manifest Delivery Checklist" : "Truck Load Checklist"}
  </span>
  <ul className="space-y-2">
    {order.items.map((item) => {
      const isChecked = !(itemChecks[order.id]?.[item.id]);
      const isDeliveringPhase = order.status === "DELIVERED";

      return (
        <li key={item.id} className="flex justify-between items-center text-slate-700 py-1.5
          border-b border-slate-200/50 last:border-0 last:pb-0">
          <span className="font-medium flex items-center gap-2.5 min-w-0">
            {isDeliveringPhase && (
              <input
                type="checkbox"
                checked={isChecked}
                onChange={e => {
                  setItemChecks((prev) => ({
                    ...prev,
                    [order.id]: {
                      ...(prev[order.id] || {}),
                      [item.id]: e.target.checked,
                    },
                  ));
                }}
              className="h-4.5 w-4.5 rounded-md border-[#E2E8F0] text-blue-600 focus:ring-blue-500 cursor-pointer shrink-0 transition-all"
            </>

```

```

    ))
    <span className={isChecked && isDeliveringPhase ? "line-through text-slate-400
font-semibold truncate" : "font-semibold text-slate-700 truncate"}>
      {item.name}
      {item.detail && <span className="text-slate-450 block text-[10px] font-medium mt-
0.5">{item.detail}</span>}
    </span>
  </span>
  <span className="font-mono font-bold text-white bg-slate-800 px-1.5 py-0.5 rounded
text-3xs shrink-0 ml-2">
    x {item.quantity}
  </span>
</li>
);
}}
</ul>
</div>

{/* Step Timeline */}
{renderTimeline(order.status)}

{/* CTA button */}
{action && (
  <div className="space-y-2 mt-4">
    {order.status === "DELIVERED" && !order.items.every(item =>
!!itemChecks[order.id]?.[item.id]) && (
      <div className="p-3 bg-amber-50 border border-amber-250 text-amber-800 rounded-2xl
text-[10px] font-bold text-center flex items-center justify-center gap-1.5 mt-4">
        <span> Delivery Checklist Required: Please review and check off all items before
initiating recovery.</span>
      </div>
    )}
    <button
      onClick={() => {
        if (order.status === "DELIVERED") {
          const allChecked = order.items.every(item => !!itemChecks[order.id]?.[item.id]);
          if (!allChecked) {
            triggerAlert("Safety Lock", "Please check off all manifest equipment lines before
initiating recovery.");
          }
          return;
        }
      }}
      triggerConfirm(
        "Phase Transition Safety Check",
        `Are you sure you want to proceed with: "${action.label}"?`,
        () => handleStatusChange(order.id, action.nextStatus)
      );
    }
    disabled={updatingId === order.id || (order.status === "DELIVERED" &&
!order.items.every(item => !!itemChecks[order.id]?.[item.id]))}
    className={`w-full py-4 text-white font-bold rounded-xl text-xs flex items-center justify-
center gap-2 active:scale-95 transition-all shadow-md ${
      (order.status === "DELIVERED" && !order.items.every(item =>
!!itemChecks[order.id]?.[item.id]))
        ? "bg-slate-350 border border-slate-350 text-slate-400 cursor-not-allowed shadow-none"
        : action.style
    }
  </div>

```

```

    }}
  >
    {updatingId === order.id ? (
      <
        <svg className="animate-spin -ml-1 mr-2 h-4 w-4 text-white" fill="none" viewBox="0
0 24 24">
          <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor"
strokeWidth="4" />
          <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373 0 0
5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042 1.135 5.824 3 7.938l3-2.647z" />
        </svg>
        Communicating with Operations Broker...
      </>
    ) : (
      action.label
    )}
  </button>
</div>
)}

{/* Undo/Previous Phase button to recover from misclicks */}
{!isFinished && getPreviousStatus(order.status) && (
  <button
    onClick={() => {
      const prev = getPreviousStatus(order.status);
      if (!prev) return;
      triggerConfirm(
        "Phase Reversion Warning",
        `Are you sure you want to undo and revert this order back to the previous phase:
"${prev.label.replace("↩", "")}"?`,
        () => handleStatusChange(order.id, prev.prevStatus)
      );
    }}
    disabled={updatingId === order.id}
    className="w-full py-2.5 bg-slate-100 hover:bg-slate-200 text-slate-600 font-bold rounded-
xl text-3xs uppercase tracking-widest flex items-center justify-center gap-1.5 mt-2.5 transition-all
active:scale-95 border border-slate-200"
  >
    Undo: {getPreviousStatus(order.status)?.label}
  </button>
)}

{/* Locked Completed notification banner */}
{isFinished && (
  <div className="bg-emerald-50 border border-emerald-200 text-emerald-700 p-3.5 rounded-
xl text-xs font-bold flex items-center justify-center gap-2 mt-4 text-center">
    <svg className="h-4 w-4 shrink-0 text-emerald-600" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={3} d="M5 13l4 4L19
7" />
    </svg>
    Logs persisted & locked in database.
  </div>
)}
</div>

```

```

    );
  }}
</div>
})
</main>

{/* Custom Royal Blue Brand Modal Popup Dialog (Replaces native alerts/confirm) */}
{modal.isOpen && (
  <div className="fixed inset-0 z-50 bg-[#0B2545]/60 backdrop-blur-sm flex items-center justify-
center p-4 animate-in fade-in duration-200">
    <div className="bg-white border border-[#E2E8F0] rounded-3xl shadow-2xl p-6 max-w-sm w-full
animate-in zoom-in-95 duration-200 flex flex-col gap-4">
      <div className="flex items-center gap-3 border-b border-slate-100 pb-3">
        <span className={`h-8 w-8 rounded-full flex items-center justify-center text-sm ${
          modal.type === "confirm" ? "bg-amber-50 text-amber-600 border border-amber-100" : "bg-red-
50 text-red-600 border border-red-100"
        }`}>
          {modal.type === "confirm" ? " ? " : " ⚠️ "}
        </span>
        <h3 className="font-extrabold text-slate-800 text-sm">{modal.title}</h3>
      </div>

      <p className="text-xs text-slate-500 font-semibold leading-relaxed">
        {modal.message}
      </p>

      <div className="flex gap-2.5 mt-2.5 justify-end">
        {modal.type === "confirm" && (
          <button
            onClick={() => setModal(prev => ({ ...prev, isOpen: false })))}
            className="px-4 py-2 bg-slate-50 border border-slate-200 hover:bg-slate-100 rounded-xl text-
slate-650 text-3xs font-extrabold uppercase tracking-wider transition-all cursor-pointer"
          >
            Cancel
          </button>
        )}
        <button
          onClick={() => {
            setModal(prev => ({ ...prev, isOpen: false }));
            if (modal.onConfirm) modal.onConfirm();
          }}
          className="px-4.5 py-2.5 bg-blue-600 hover:bg-blue-700 text-white rounded-xl text-3xs font-
extrabold uppercase tracking-wider transition-all active:scale-95 shadow cursor-pointer flex items-center
justify-center"
        >
          {modal.type === "confirm" ? "Confirm" : "Understood"}
        </button>
      </div>
    </div>
  </div>
)}
</div>
);
}

```